MIT 8103 Advanced Database Systems

MediStock NG Hospital Pharmacy Inventory · Samuel Olalekan Afolabi · Student ID 301929738

Case study

My case study is MediStock NG Hospital Pharmacy Inventory. I designed the database for pharmacists, inventory officers and hospital auditors, with separate tables named suppliers, medicines, pharmacies, inventory_batches and dispensing_events. It supports these operations: register medicines and suppliers; receive and monitor stock batches; dispense medicines and record batch usage; review low-stock and expiry reports.

Portfolio 1: Database Design and Modelling

I organised the relational design into 5 tables. Primary keys identify records, foreign keys preserve the links, and checks enforce the stated business rules. Inventory batch quantity_on_hand must be zero or greater. The normalisation examples show the sample data at 1NF, 2NF and 3NF. All 19 deliberately invalid inserts were rejected without changing the valid sample rows.

[image]

Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries against the sample records. The join returned 1 row and included AMX-2026-001 with Amoxicillin. For the index comparison, I used 25,000 separate performance rows. The median lookup time changed from 1.147 ms to 0.195 ms. Both phases returned the same records. The measured plan used the new index. These timings apply to this lookup and environment.

Portfolio 3: Transactions and Concurrency

I used a temporary counter on inventory_batches to test COMMIT, ROLLBACK and READ COMMITTED isolation. The uncommitted update remained hidden from the other session, and ROLLBACK restored the earlier committed value. Without locking, two increments left the counter at 2. With SELECT FOR UPDATE, Session B waited for Session A and the final value was 3. The helper is confined to this exercise; the case's original business fields stay unchanged.

Portfolio 4: NoSQL and Advanced Data Models

I used pharmacy_inventory_events for the case's event documents. MongoDB inserted 2 sample events and returned the expected results for 4 queries. Validation rejected an invalid date type and a duplicate event ID. Embedded snapshots keep related details with an event, while the details field can differ by event type. These are fictional events; PostgreSQL continues to enforce its relational rules.

Repository and evidence

https://github.com/afosam/Afolabi_Samuel_301929738

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed three MongoDB replica members across separate failure zones. The discussion covers primary elections, majority acknowledgement, secondary lag and sharding trade-offs for pharmacy_inventory_events. Replication supports availability, while backups address recovery. PostgreSQL needs its own availability plan. This is the permitted discussion option; no live replica set or failover test was deployed.

Individual reflection

I kept suppliers, medicines, pharmacies, inventory_batches, dispensing_events in separate tables so their details could be maintained without repeating them in every related record. Inventory batch quantity_on_hand must be zero or greater. The constraint tests check this alongside the required values and record links. One issue illustrated by the two-session exercise is that both sessions can read the same value before either update is committed. Locking the row before reading preserves both increments. The index comparison also shows why I need to check the execution plan as well as the final answer: the same result can require different amounts of database work.

AI-use declaration

AI Use Declaration

I used GitHub Copilot and an AI-assisted portfolio generator during the preparation of this assignment. These tools helped me develop and refine the database models, SQL and MongoDB scripts, and some of the technical explanations.

The portfolio generator was also used to run the PostgreSQL and MongoDB exercises, check the results and capture screenshots showing the execution outputs.

I reviewed the generated materials and results to ensure they were consistent with the assignment requirements. I take responsibility for the final submission, including the database designs, implementation decisions, analysis and explanations. I also remain responsible for understanding the work presented and being able to explain the processes and results.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams